

Compiladores

Juliana da Costa Feitosa
juliana.feitosa@unesp.br
Docente UNESP - Bauru-SP

Aula 4

Revisão de conceitos de gramáticas

Introdução

- Alfabeto: Σ
- Palavra: w
- Comprimento de palavra: $|w|$
- Palavra vazia: λ ou ϵ
- Fechamento: Σ^+ ou Σ^*

Linguagens

- Uma linguagem é uma forma de expressar ideias através do uso de símbolos, sejam eles escritos, orais ou de outro tipo, que sigam as regras específicas

Linguagens

- Uma **linguagem (L)** sobre um alfabeto Σ é um subconjunto de Σ^* ($L \subset \Sigma^*$)
- **Exemplo:**
 - $\Sigma = \{0, 1\}$
 - $L = \{\epsilon, 00, 01, 10, 11, 0000, 0001, \dots, 1111, \dots, 000000, 000001, \dots\}$

Linguagens

- **Exemplo:**
 - Σ = Alfabeto Pascal
 - A linguagem Pascal P é um subconjunto de Σ^* ($P \subset \Sigma^*$ e $P \neq \Sigma^*$)

Linguagens

- Exemplo:

- $\emptyset$ e $\{\epsilon\}$ é uma linguagem
- $\Sigma = \{0, 1\}$
 - O conjunto de **palíndromos** pode ser considerado uma linguagem
 - 010 é uma palavra

Linguagens

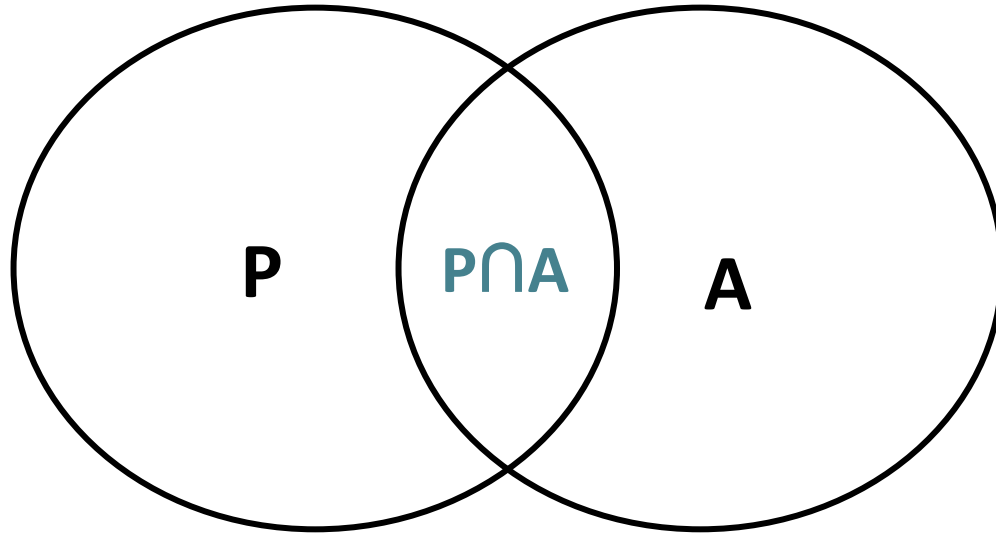
- Exemplo:

- Supondo que desejamos eliminar a palavra reservada **GOTO** dos programas Pascal, a nova linguagem seria:

- $A = \Sigma^* - \{\text{GOTO}\}$

- $L = P \cap A$ ($P = \text{Linguagem Pascal}$)

Linguagens



Linguagens

- Exemplo:

- Suponha que L seja uma linguagem sobre $\Sigma = \{0,1\}$, cujas palavras terminem com **1** ou com **00**
- $L = \Sigma^* \cdot \{1, 00\}$
- **Ao final de cada palavra, será concatenado o 1 e 00**

Linguagens

- Exemplo:

- Suponha que L seja uma linguagem sobre $\Sigma = \{0,1\}$, cujas palavras são números binários pares (**terminam com 0**)
- $L = \Sigma^* \cdot \{0\}$
- **Ao final de cada palavra, será concatenado o 0**

Linguagens

- Para escrevermos palavras em nossa língua portuguesa, é preciso seguir **regras gramaticais** para que elas façam sentido
 - **“aaaa12”** apesar de usar símbolos da língua portuguesa e ser uma **palavra possível**, não constitui uma **palavra válida**

Gramáticas

- Uma **gramática**, de maneira **informal**, pode ser definida como sendo:
 - Um sistema gerador de linguagens;
 - Um sistema de reescrita;
 - Uma maneira finita de descrever (representar) uma linguagem;
e
 - Um dispositivo formal usado para especificar de maneira finita e precisa uma linguagem potencialmente infinita.

Gramáticas

- Uma **gramática**, de maneira **formal**, é um conjunto finito de regras que, quando aplicadas em sequência, geram palavras. Ela também é uma quádrupla ordenada $G = (V_N, V_T, P, S)$
 - V_N ou **N**: conjunto de símbolos não terminais (letras maiúsculas ou nomes mnemônicos entre <>)
 - V_T ou **T**: conjunto de símbolos terminais (letras minúsculas, símbolos especiais e compostos como begin)
 - **P**: conjunto de regras de produção ou de transcrição
 - **S**: símbolo inicial

Gramáticas

- Se fôssemos representar a gramática da língua portuguesa, temos $G = (V_N, V_T, P, S)$, onde:
 - V_N : {<sentença>, <sujeito>, <predicado>, <substantivo>, <artigo>, <adjetivo>, <predicado>, <verbo>, <objeto>}
 - V_T : {joão, maria, cachorro, livro, pão, o, a, pequeno, bom, bela, morde, lê, olha}
 - **P**: é o conjunto das regras gramaticais apresentado
 - **S**: <sentença>

Gramáticas

- V_N e V_T devem ser disjuntos, ou seja, não deve existir nenhum elemento comum entre eles ($V_N \cap V_T = \emptyset$)
- $P \subseteq V^+ \times V^*$
- $V = V_N \cup V_T$
- $S \in V_N$

Gramáticas

- Fazendo uma analogia entre o exemplo intuitivo e a noção formal de gramática, constatamos que:
 - V_N : são as categorias sintáticas ou gramaticais;
 - V_T : são as palavras utilizadas como símbolos da linguagem;
 - P : são as regras sintáticas (ou gramaticais); e
 - S : é a categoria gramatical que sintetiza o que será produzido (gerado) pela gramática.

Gramáticas

- Exemplo 1:

- $G = (V_N, V_T, P, S)$
- $V_N : \{E, T, F\}$
- $V_T : \{a, b, +, *\}$
- $P : \{(E \rightarrow E+T), (E \rightarrow T), (T \rightarrow T*F), (T \rightarrow F), (F \rightarrow a), (F \rightarrow b)\}$
- $S : E$

- Exemplo 2:

- $G = (V_N, V_T, P, S)$
- $V_N : \{S\}$
- $V_T : \{a, b\}$
- $P : \{(S \rightarrow aSb), (S \rightarrow \epsilon)\}$
- $S : S$

Gramáticas

```
#include <stdio.h>
#include <iso646.h>

void main(){

    int a = 1, b=2;

    if(a==0 and b==0)
        printf("A e B sao zero");
    else if (a==1 or b==1)
        printf("Pelo menos um valor eh um");
    else if (a==2 && b==2)
        printf("A e B sao iguais a dois");
}
```

Expressões

Comandos

NÃO TERMINAIS

Gramáticas

```
program test(input,output);  
var  
  i:integer;  
begin  
  read(i);  
  if i<10 then  
    writeln('Your input is smaller than 10')  
  else  
    writeln('Your input is larger than or equal  
end.  
[.] Watches  
i: 11
```

Expressão

Comandos

NÃO TERMINAIS

Gramáticas

- É possível realizar dois tipos de operações, de acordo com as regras de produção de uma gramática
 - **Derivação**
 - **Redução**
- **Derivação** é a operação adequada para geração de sentenças, enquanto que a operação de **Redução** é adequada ao reconhecimento de sentenças.

Derivação

- **Derivação** é a operação que consiste em substituir uma string (ou parte dela) por outra, de acordo com as produções das gramáticas em questão, no sentido símbolo **símbolo inicial** $\rightarrow$ **sentença** (também usamos o símbolo $::=$)
- **Exemplo:**
 - $aAb \rightarrow aabb$

Derivação

- **Exemplo:** $G = (V_N, V_T, P, S)$
 - $V_N : \{S\}$
 - $V_T : \{a, b\}$
 - $P : \{(S \rightarrow aSb), (S \rightarrow \epsilon)\}$
 - $S : S$
 - $aaSbb \rightarrow aaaSbbb$

Derivação

- Uma cadeia de símbolos nem sempre é derivada diretamente
- **Exemplo:** Consideremos a gramática $G = (\{S\}, \{a,b\}, \{S ::= ab \mid aSb\}, S)$
 - $S \rightarrow ab$
 - $S \rightarrow aSb \rightarrow aabb$
 - $S \rightarrow aSb \rightarrow aaSbb \rightarrow aaabbb$
- **aaabbb** é derivável de **S**, porém não diretamente

Redução

- **Redução** é a operação que consiste na substituição de uma string (ou parte dela) por outro, de acordo com as produções da gramática, no sentido **sentença** $\rightarrow$ **símbolo inicial**.
- **Exemplo:**
 - $aabb \rightarrow aAb$

Redução

- Exemplo: $G = (V_N, V_T, P, S)$
 - $V_N : \{S\}$
 - $V_T : \{a, b\}$
 - $P : \{(S \rightarrow aSb), (S \rightarrow \epsilon)\}$
 - $S : S$
 - $aa**S**bbb \rightarrow aa**S**bb$

Linguagem gerada por uma gramática

- Uma **linguagem** é definida pelo conjunto de sentenças que podem ser derivadas a partir do **símbolo inicial da gramática** que a representa.
- Formalmente definimos a linguagem gerada por $G = (V_n, V_t, P, S)$, denotada por $L(G)$, como sendo: $L(G) = \{x \mid x \in V_t^* \wedge Sx \rightarrow x\}$;

Linguagem gerada por uma gramática

- Uma **linguagem** é definida pelo conjunto de sentenças que podem ser derivadas a partir do **símbolo inicial da gramática** que a representa.
- Formalmente definimos a linguagem gerada por $G = (V_n, V_t, P, S)$, denotada por $L(G)$, como sendo: $L(G) = \{x \mid x \in V_t^* \wedge Sx \rightarrow x\}$;

Produce x , tal que x pertence aos símbolos terminais, e existe uma regra de produção que do símbolo inicial da gramática eu chegue em x

Linguagem gerada por uma gramática

- É o conjunto de todas as cadeias formadas apenas por símbolos **terminais** e que são deriváveis em G a partir da cadeia formada pelo símbolo inicial da gramática
- O objetivo é transformar os **não-terminais em terminais** por meio das regras de produção

Linguagem gerada por uma gramática

- Exemplo 1: $G = (V_N, V_T, P, S)$
 - $V_N : \{S, T, W\}$
 - $V_T : \{a, b\}$
 - $P : \{(S \rightarrow aT), (T \rightarrow bW), (W \rightarrow bT), (W \rightarrow \epsilon)\}$
 - $S : S$
 - $L(G) = \{ab, abbb, abbbbb....., \}$
 - $L(G) = \{a.b^n, n = 2n+1, n \in \mathbb{N}\}$

Linguagem gerada por uma gramática

Exemplo 1: $G = (V_N, V_T, P, S)$

$V_N : \{S, T, W\}$

$V_T : \{a, b\}$

$P : \{(S \rightarrow aT), (T \rightarrow bW), (W \rightarrow bT), (W \rightarrow \epsilon)\}$

$S : S$

$L(G) = \{ab, abbb, abbbbb, \dots, \}$

$L(G) = \{a.b^n, n = 2n+1, n \in \mathbb{N}\}$

1 - $S \rightarrow aT \rightarrow abW \rightarrow ab$

2 - $S \rightarrow aT \rightarrow abW \rightarrow abbT \rightarrow abbbW \rightarrow abbb$

3 - $S \rightarrow aT \rightarrow abW \rightarrow abbT \rightarrow abbbW \rightarrow abbbbW \rightarrow abbbbbT \rightarrow abbbbbbW \rightarrow abbbbbb$

Linguagem gerada por uma gramática

- **Exemplo 2:** $G = (V_N, V_T, P, S)$
 - $V_N : \{S\}$
 - $V_T : \{a\}$
 - $P : \{(S \rightarrow aS), (S \rightarrow \epsilon)\}$
 - $S : S$
 - $L(G) = \{\epsilon, a, aa, aaa, aaaa, \dots, \}$
 - $L(G) = a^*$

Linguagem gerada por uma gramática

Exemplo 2: $G = (V_N, V_T, P, S)$

$V_N : \{S\}$

$V_T : \{a\}$

$P : \{(S \rightarrow aS), (S \rightarrow \epsilon)\}$

$S : S$

$L(G) = \{\epsilon, a, aa, aaa, aaaa, \dots, \}$

$L(G) = a^*$

1 - $S \rightarrow \epsilon$

2 - $S \rightarrow aS \rightarrow a$

3 - $S \rightarrow aS \rightarrow aaS \rightarrow aa$

4 - $S \rightarrow aS \rightarrow aaS \rightarrow aaaS \rightarrow aaa$

Linguagem gerada por uma gramática

- **Exemplo 3:** $G = (V_N, V_T, P, S)$
 - $V_N : \{S, T\}$
 - $V_T : \{a, b\}$
 - $P : \{(S \rightarrow aT), (T \rightarrow bS), (S \rightarrow \epsilon)\}$
 - $S : S$
 - $L(G) = \{\epsilon, ab, abab, ababab, \dots, \}$
 - $L(G) = (a.b)^*$

Linguagem gerada por uma gramática

Exemplo 3: $G = (V_N, V_T, P, S)$

$V_N : \{S, T\}$

$V_T : \{a, b\}$

$P : \{(S \rightarrow aT), (T \rightarrow bS), (S \rightarrow \epsilon)\}$

$S : S$

$L(G) = \{\epsilon, ab, abab, ababab, \dots, \}$

$L(G) = (a.b)^*$

1 - $S \rightarrow \epsilon$

2 - $S \rightarrow aT \rightarrow abS \rightarrow ab$

3 - $S \rightarrow aT \rightarrow abS \rightarrow abaT \rightarrow ababS \rightarrow abab$

4 - $S \rightarrow aT \rightarrow abS \rightarrow abaT \rightarrow ababS \rightarrow ababaT \rightarrow abababS \rightarrow ababab$

Linguagem gerada por uma gramática

- **Exemplo 4:** $G = (V_N, V_T, P, S)$
 - $V_N : \{S, T\}$
 - $V_T : \{a, b\}$
 - $P : \{(S \rightarrow aS), (S \rightarrow aT), (S \rightarrow a), (T \rightarrow bT), (T \rightarrow \epsilon)\}$
 - $S : S$
 - $L(G) = \{a, aa, aaa, \dots, ab, abb, \dots, aa^n b^n\}$
 - $L(G) = a.a^*b^*$

Linguagem gerada por uma gramática

Exemplo 4: $G = (V_N, V_T, P, S)$

$V_N : \{S, T\}$

$V_T : \{a, b\}$

$P : \{(S \rightarrow aS), (S \rightarrow aT), (S \rightarrow a), (T \rightarrow bT), (T \rightarrow \epsilon)\}$

$S : S$

$L(G) = \{a, aa, aaa, \dots, ab, abb, \dots, aa^n b^n\}$

$L(G) = a.a^*b^*$

$S \rightarrow a$

$S \rightarrow aS \rightarrow aa$

$S \rightarrow aS \rightarrow aaS \rightarrow aaa$

...

$S \rightarrow aT \rightarrow abT \rightarrow ab$

$S \rightarrow aT \rightarrow abT \rightarrow abbT \rightarrow abb$

Linguagem gerada por uma gramática

- **Exemplo 5:** $G = (V_N, V_T, P, S)$
 - $V_N : \{S, T\}$
 - $V_T : \{a, b\}$
 - $P : \{(S \rightarrow aS), (T \rightarrow bT), (S \rightarrow bT), (S \rightarrow \epsilon), (T \rightarrow \epsilon)\}$
 - $S : S$
 - $L(G) = \{\epsilon, a, aa, aaa, b, bb, bbb, ab, aab, \dots, \}$
 - $L(G) = a^*.b^*$

Linguagem gerada por uma gramática

Exemplo 5: $G = (V_N, V_T, P, S)$

$V_N : \{S, T\}$

$V_T : \{a, b\}$

$P : \{(S \rightarrow aS), (T \rightarrow bT), (S \rightarrow bT), (S \rightarrow \epsilon), (T \rightarrow \epsilon)\}$

$S : S$

$L(G) = \{\epsilon, a, aa, aaa, b, bb, bbb, ab, .aab, \dots, \}$

$L(G) = a^* . b^*$

1 - $S \rightarrow \epsilon$

2 - $S \rightarrow aS \rightarrow a$

3 - $S \rightarrow aS \rightarrow aaS \rightarrow aa$

4 - $S \rightarrow aS \rightarrow aaS \rightarrow aabT \rightarrow aab$

5 - $S \rightarrow bT \rightarrow b$

6 - $S \rightarrow bT \rightarrow bbT \rightarrow bb$

Expressões aritméticas

- Seja a gramática G abaixo, dada por suas regras
 - $E \rightarrow E + T \mid T$
 - $T \rightarrow T * F \mid F$
 - $F \rightarrow (E) \mid a$
 - Considere a cadeia $x = a*(a+a)+a$

Expressões aritméticas

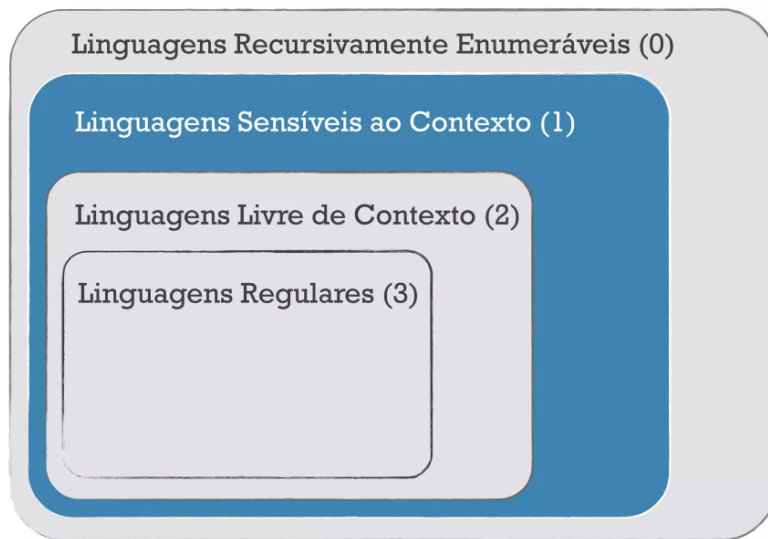
- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid a$
- Considere a cadeia $x = a^*(a+a)+a$
 - $E \rightarrow E+T \rightarrow T+T \rightarrow T*F+T \rightarrow F*F+T \rightarrow a*F+T \rightarrow a*(E)+T \rightarrow$
 $a*(E+T)+T \rightarrow a*(T+T)+T \rightarrow a*(F+T)+T \rightarrow a*(a+T)+T \rightarrow a*(a+F)+T$
 $\rightarrow a*(a+a)+T \rightarrow a*(a+a)+F \rightarrow a*(a+a)+a$

Tipos de Gramática

- Gramáticas Irrestrita ou Recursivamente Enumeráveis - GEF - Tipo 0
- Gramáticas Sensíveis ao Contexto - GSC - Tipo 1
- Gramáticas Livres de Contexto - GLC - Tipo 2
- Gramáticas Regulares - GR - Tipo 3

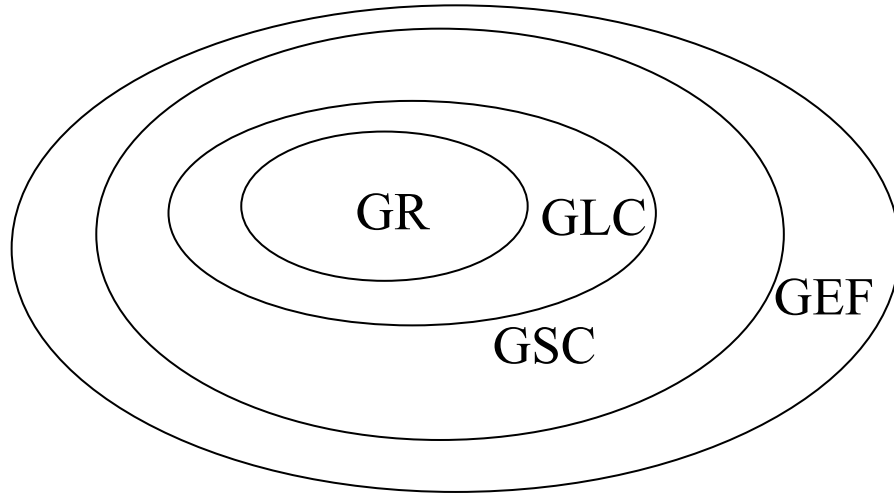
Tipos de Gramática

Hierarquia de Chomsky



Tipos de gramática

- Hierarquia de CHOMSKY



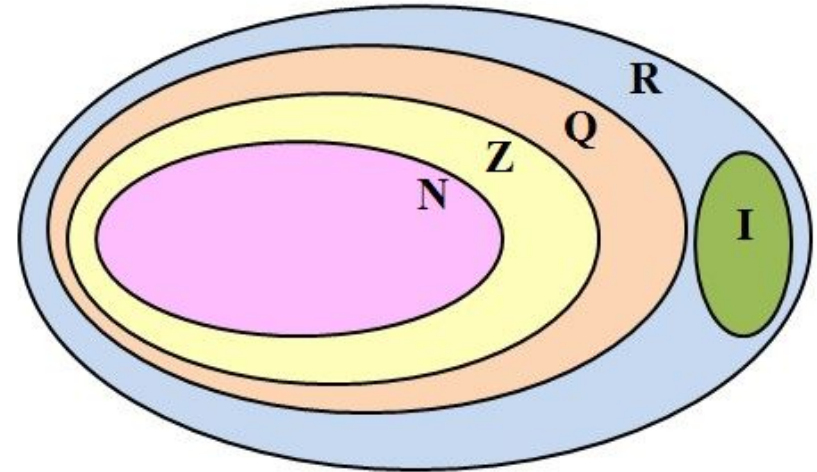
N - Naturais (1,2,3)

Z - Inteiros (-1, -2, -3)

Q - Racionais ($-\frac{1}{3}$, 0,333)

I - Irracional (Pi)

R - Reais (Todos os números)



Tipos de Gramática

Gramáticas	Linguagens	Reconhecedores
Regulares	Regulares	Autômatos Finitos
Livres de Contexto	Livres de Contexto	Autômatos com Pilhas
Sensíveis ao Contexto	Sensíveis ao Contexto	Autômatos Linearmente Limitados
Irrestrita	Recursivamente Enumeráveis	Máquina de Turing

Tipos de gramática

- Gramáticas do tipo 3- GR

- Seja $G (V_N, V_T, P, S)$ uma gramática onde toda produção é da forma $A \rightarrow aB$ ou $A \rightarrow b$, onde:
 - $A, B \in V_N$
 - $a \in V_T$
 - $b \in V_T \cup \{\lambda\}$
- Parte esquerda fixa como não terminal, e parte direita possui **um** terminal ou **um** terminal junto de **um** não terminal

Tipos de gramática

- Gramáticas do tipo 3 - GR

- Exemplo: $G: (V_N, V_T, P, S)$, $V_N = \{S, A, B\}$, $V_T = \{a, b\}$

- As regras de produção P :

- $S \rightarrow aB$

- $A \rightarrow a$

- $S \rightarrow \lambda$

- $A \rightarrow \lambda$ (Não pode)

- Apenas o símbolo inicial pode gerar λ e ele não pode aparecer do lado direito das produções

Tipos de gramática

- Gramáticas do tipo 2 - GLC

- Seja $G (V_N, V_T, P, S)$ uma gramática onde toda produção é da forma $A \rightarrow \beta$, onde:
 - $A \in V_N$
 - $\beta \in (V_N \cup V_T)^*$
- Parte esquerda fixa como não terminal, e parte direita pode ser terminal ou não terminal

Tipos de gramática

- Gramáticas do tipo 2 - GLC

- Exemplo: $G: (V_N, V_T, P, S)$, $V_N = \{S, A, B\}$, $V_T = \{a, b\}$

- As regras de produção P :

- $S \rightarrow aB$
 - $S \rightarrow ba$
 - $A \rightarrow a$
 - $A \rightarrow bAS$
 - $B \rightarrow S$
 - $S \rightarrow \lambda$
 - $A \rightarrow \lambda$

Tipos de gramática

- Gramáticas do tipo 1 - GSC

- Seja $G (V_N, V_T, P, S)$ uma gramática onde toda produção é da forma $\alpha \rightarrow \beta$ tal que $|\beta| \geq |\alpha|$
 - $\alpha \in (V_N \cup V_T)^+$
 - $\beta \in (V_N \cup V_T)^*$
- Tanto a parte esquerda quanto a parte direita podem ser terminal ou não terminal

Tipos de gramática

A parte direita tem que ser maior ou igual do que da parte esquerda

- Gramáticas do tipo 1 - GSC

- Seja $G (V_N, V_T, P, S)$ uma gramática onde toda produção é da forma $\alpha \rightarrow \beta$ tal que $|\beta| \geq |\alpha|$
 - $\alpha \in (V_N \cup V_T)^+$
 - $\beta \in (V_N \cup V_T)^*$
- Tanto a parte esquerda quanto a parte direita podem ser terminal ou não terminal

Tipos de gramática

- Gramáticas do tipo 1 - GSC

- Exemplo: $G: (V_N, V_T, P, S)$, $V_N = \{S, B, C\}$, $V_T = \{a, b, c\}$

- As regras de produção P :

- $S \rightarrow aSBC$
 - $S \rightarrow aBC$
 - $CB \rightarrow BC$
 - $aB \rightarrow ab$
 - $bB \rightarrow bb$
 - $S \rightarrow \lambda$ e $A \rightarrow \lambda$ não pode - $|\lambda| = 0$

Tipos de gramática

- **Gramáticas do tipo 0 - GEF**

- Para este tipo de gramática as produções são da forma $\alpha \rightarrow \beta$, onde
 - $\alpha \in V^+$
 - $\beta \in V^*$
- Tanto a parte esquerda quanto a parte direita podem ser terminal ou não terminal

Tipos de gramática

- Gramáticas do tipo 0 - GEF

- Exemplo: $G: (V_N, V_T, P, S)$, $V_N = \{S, B, C\}$, $V_T = \{a, b, c\}$

- As regras de produção P :

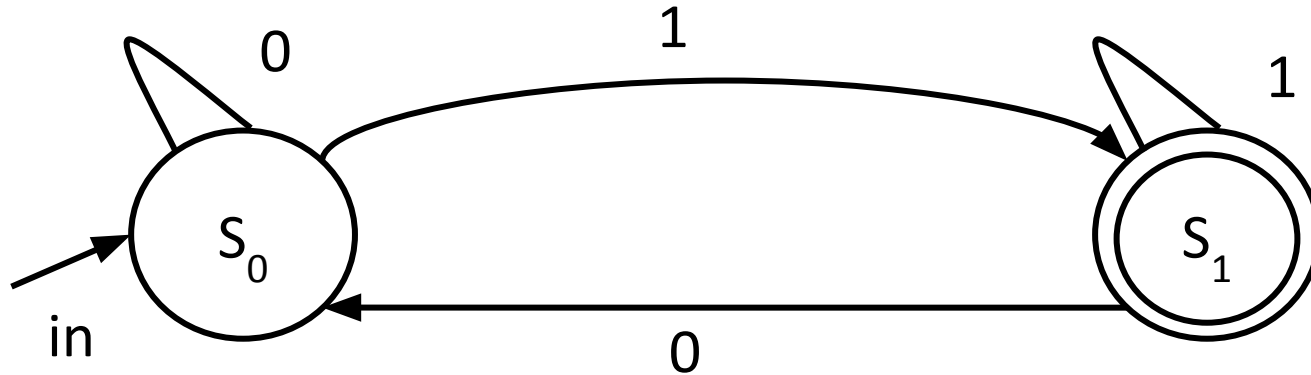
- $SabCa \rightarrow aSBC$
 - $SAB \rightarrow aB$
 - $CBa \rightarrow BC$
 - $aB \rightarrow abb$
 - $bB \rightarrow bb$
 - $A \rightarrow \lambda$
 - $S \rightarrow \lambda$

Tipos de gramática

- As **gramáticas GR e GLC** são resolvidas por computadores
- As **gramáticas GSC e GEF** são mais complexas e por isso precisam de estruturas específicas para serem resolvidas, como por exemplo a Máquina de Turing

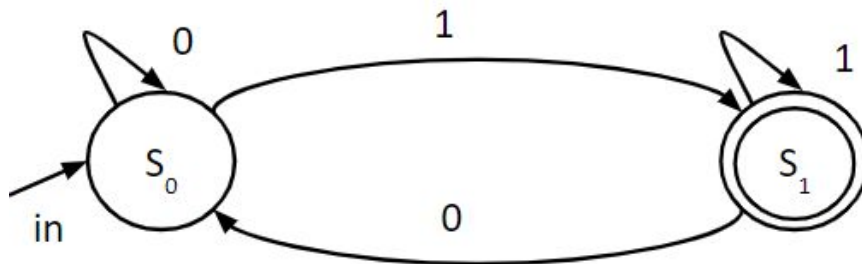
Autômatos finitos

- Seja um diagrama abaixo



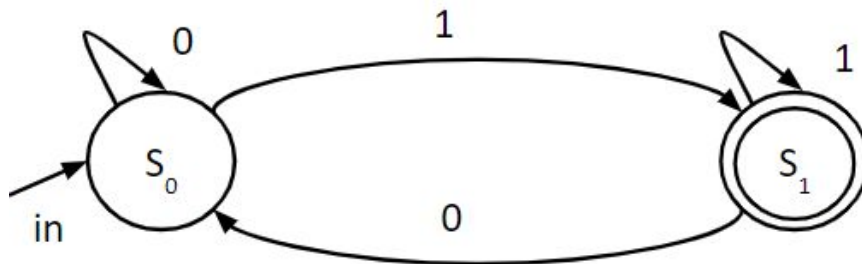
Autômatos finitos

- S_0 e S_1 são dois estados distintos
- A flecha “in” aponta o **estado inicial**
- O **estado final** é representado por dois círculos



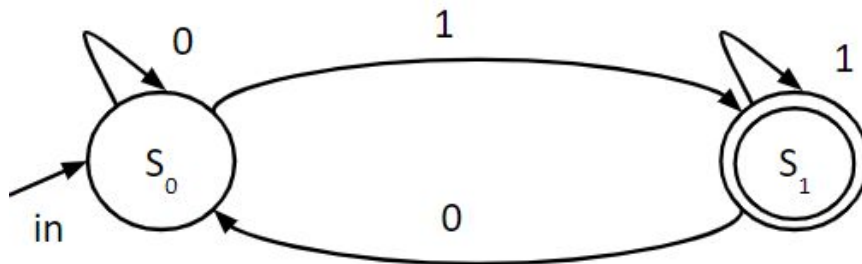
Autômatos finitos

- A máquina pode mudar de estado fazendo uma transição (de um estado S_i e S_j , sendo que nesse exemplo, $i=0$ e $j=1$)



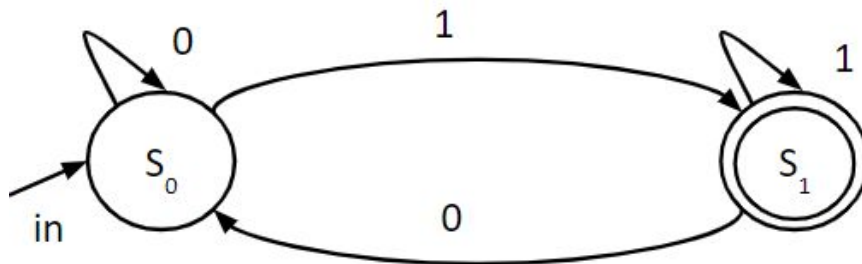
Autômatos finitos

- Assim, estando em S_0 :
 - Se a máquina ler
 - **1**: vai para o estado S_1
 - **0**: permanece no estado S_0



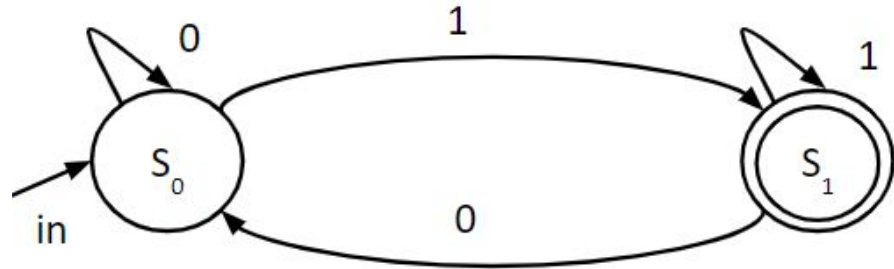
Autômatos finitos

- Por outro lado, estando em S_1 :
 - Se a máquina ler
 - **1**: permanece no estado S_1
 - **0**: vai para o estado S_0



Autômatos finitos

δ	0	1
s_0	s_0	s_1
s_1	s_0	s_1

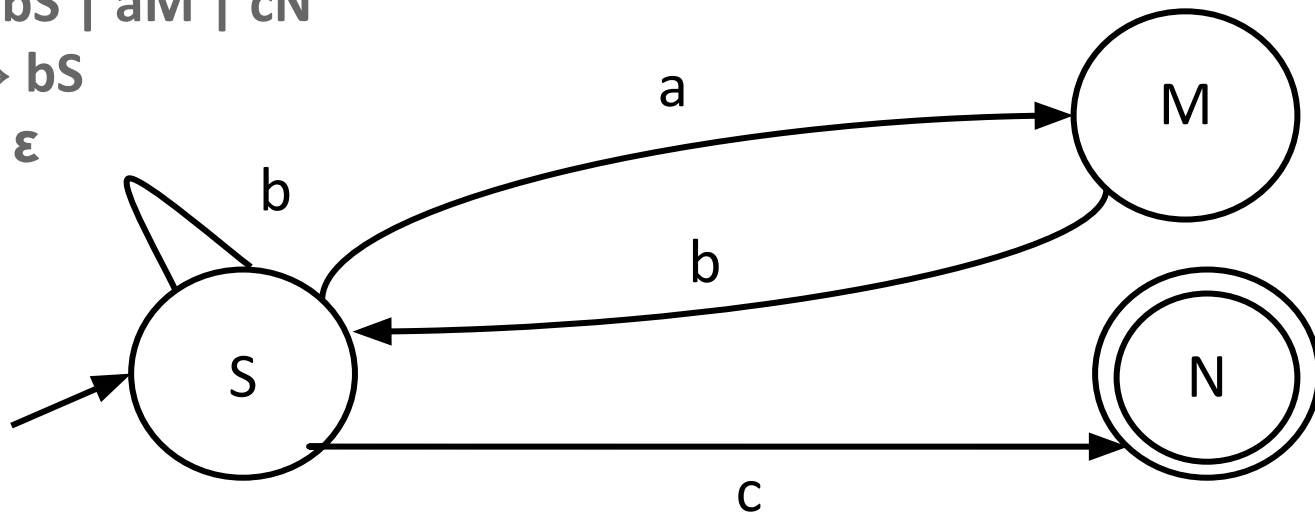


Autômatos finitos

- **Exemplo:** Seja uma $G: (V_N, V_T, P, S)$
 - $V_N = \{S, M, N\}$
 - $V_T = \{a, b, c\}$
 - $S: S$
 - $S \rightarrow bS \mid aM \mid cN$
 - $M \rightarrow bS$
 - $N \rightarrow \epsilon$

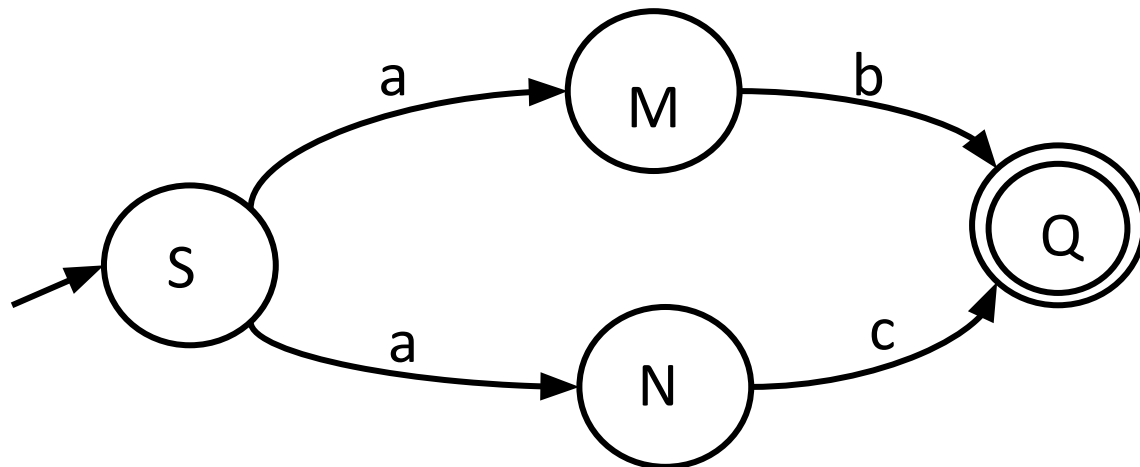
Autômatos finitos

- Exemplo: Seja uma $G: (V_N, V_T, P, S)$
 - $S \rightarrow bS \mid aM \mid cN$
 - $M \rightarrow bS$
 - $N \rightarrow \epsilon$



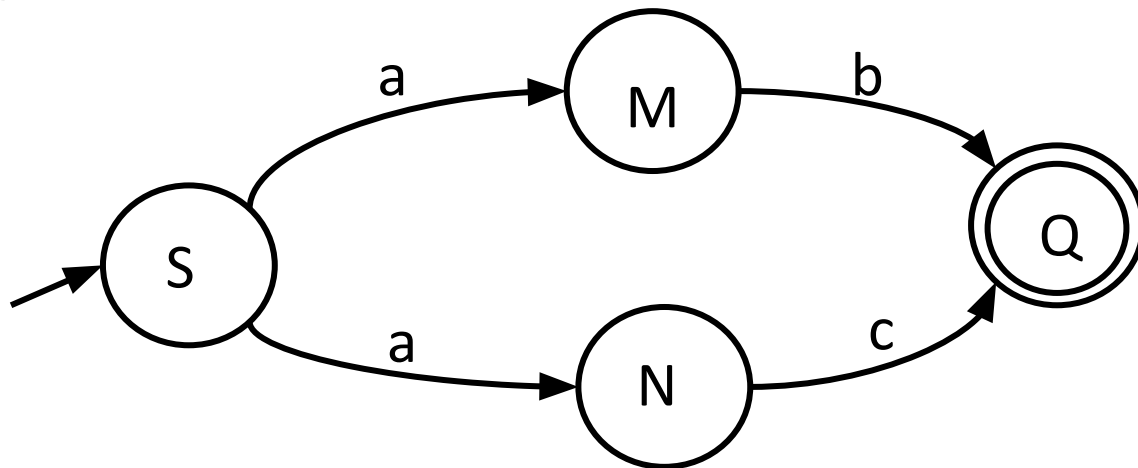
Autômatos finitos

- Exemplo 2: Seja uma $G: (V_N, V_T, P, S)$
 - $S \rightarrow aM \mid aN$
 - $M \rightarrow b$
 - $N \rightarrow c$



Autômatos finitos

- **Autômatos não-determinísticos:** a máquina escolhe um dos estados que se é permitido “entrar”
 - $S \rightarrow aM \mid aN$



Autômatos finitos

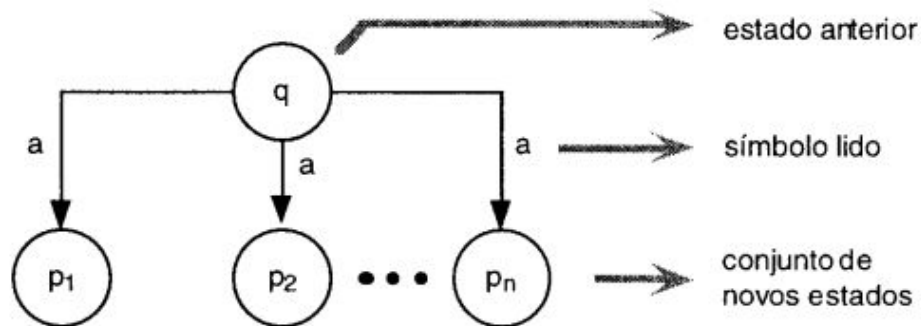
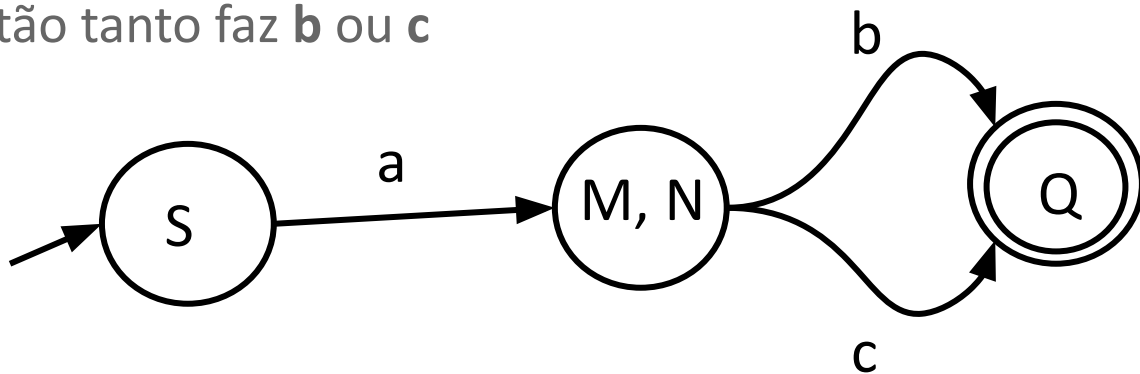


Figura 2.8 Representação da função programa não-determinística como um grafo

programa, pode-se afirmar que um Autômato Não-Determinístico assume um conjunto de estados alternativos, como se houvesse uma multiplicação da unidade de controle, uma para cada alternativa, processando independentemente, sem compartilhar recursos com as demais. Assim, o processamento de um caminho não influi no estado, símbolo lido e posição da cabeça dos demais caminhos alternativos.

Autômatos finitos

- **Autômatos determinísticos:** O caminho da máquina é completamente determinado. Nesse caso, cria-se um conjunto de estados, onde qualquer símbolo que funcione para algum estado do conjunto, funciona para o conjunto como um todo
 - $S \rightarrow aM \mid aN$: agora temos um **único estado** $\{M, N\}$. Se eu estiver em $\{M, N\}$, então tanto faz **b** ou **c**



Conjuntos regulares

- Um **Conjunto Regular** pode ser definido de 4 maneiras:
 - **Definição matemática (primitiva)**
 - Seja Σ um alfabeto qualquer. Portanto, um subconjunto de Σ^* é um CR se e somente se ele é $\varnothing$, $\{\epsilon\}$, $\{a\}$, ou pode ser obtido a partir destes através de um número finito de aplicação das operações de **União, Concatenação e Fechamento**.

Conjuntos regulares

- Um **Conjunto Regular** pode ser definido de 4 maneiras:
 - **CR** são as linguagens geradas por **Gramáticas Regulares**.
 - **CR** são as linguagens reconhecidas por **Autômatos Finitos**.
 - **CR** são os conjuntos denotados (representados) por **Expressões Regulares**.

Expressões regulares

- As **Expressões Regulares** são uma ferramenta adequada para descrever **símbolos lexicográficos**
- Dado Σ , são expressões regulares sobre Σ :
 - $\Phi, \lambda, a, \forall a \in \Sigma$
 - Se E_1 e E_2 são expressões regulares, então $(E_1 \cup E_2)$ é uma expressão regular e $(E_1 \cdot E_2)$ é uma expressão regular.

Expressões regulares

- Dada uma expressão regular E , a linguagem $L(E)$ associada a E é dada por: $L(E) \subseteq \Sigma^*$
 - $E = \Phi \Rightarrow L(E) = \Phi$
 - $E = \lambda \Rightarrow L(E) = \{\lambda\}$
 - $E = a, a \in \Sigma \Rightarrow L(E) = \{a\}$
 - $E = E_1 \cup E_2 \Rightarrow L(E) = L(E_1) \cup L(E_2)$
 - $E = E_1^* \Rightarrow L(E) = (L(E_1))^*$
 - $E = E_1.E_2 \Rightarrow L(E) = L(E_1).L(E_2)$

Regex

- É uma forma bem útil e eficiente para **coletar informações em strings**, bem como de **encontrar padrões de textos no Python**.
- **Exemplo:**
 - Verificação de e-mail e senha
 - A verificação de e-mail fora do formato ou uma senha que não tem o padrão pedido pode ser feita pelo Regex

The image shows a login page for 'TRANSFEERA'. At the top is the logo. Below it is a red banner with the text 'Usuário ou senha incorretos.' (User or password incorrect). The main content area has the text 'Que bom que voltou!' (Nice to see you back!). Below this are two input fields: the first contains 'meuemail@email.com' and the second contains masked characters '*****'. To the right of the password field is a link 'Esqueceu sua senha?' (Forgot your password?). At the bottom of the form is a blue button labeled 'Entrar' (Login). The footer contains the copyright notice '© 2020 Transfeera.com' and a link to the 'Política de privacidade' (Privacy Policy).

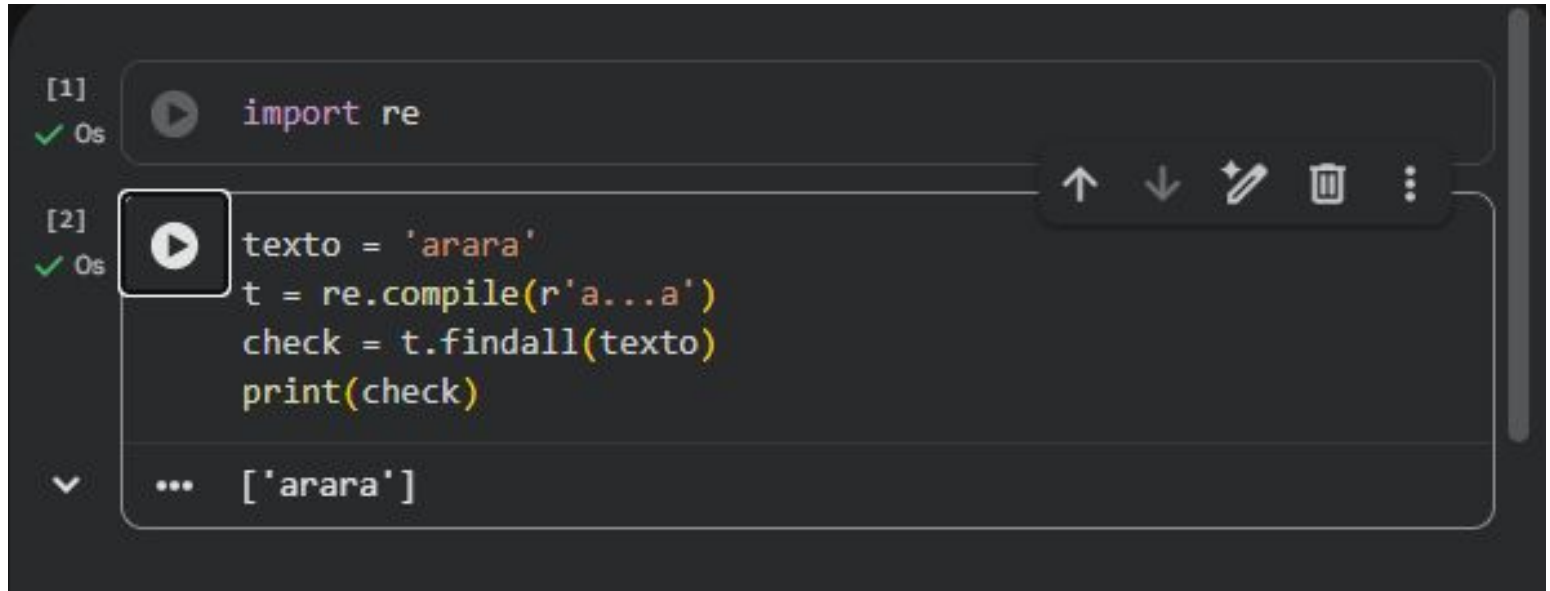
Regex

- O Regex é uma forma computadorizada de **entender strings como um conjunto de padrões**.
- **Exemplo:**
 - Para trabalhar com CPFs em um sistema, você deve partir do pressuposto de que a sequência de números tem que estar naquele formato, com vários blocos separados por um ponto.
- A **expressão regular** ajuda a definir isso e permite validar as entradas para que não haja erros.

Regex

- Elementos que possibilitam que as operações sejam executadas:
 - `[]` – conjunto de caracteres;
 - `\` – sequência especial de caracteres;
 - `^` – buscar elementos no início da string;
 - `$` – buscar elementos no final da string;
 - `*` – buscar zero ou mais repetições de uma substring;
 - `+` – uma ou mais aparições de uma substring;
 - `?` – zero ou uma aparição; e
 - `|` – busca um caractere ou outro.
- Assim, você pode criar uma **expressão completa** que **selecione elementos dentro de uma string**.

Regex



The screenshot shows a Jupyter Notebook interface with a dark theme. It contains two code cells. The first cell, labeled [1], has a green checkmark and '0s' execution time, and contains the code `import re`. The second cell, labeled [2], also has a green checkmark and '0s' execution time, and contains the code `texto = 'arara'`, `t = re.compile(r'a...a')`, `check = t.findall(texto)`, and `print(check)`. The output of the second cell is shown below it as `['arara']`. To the right of the code cells is a toolbar with icons for undo, redo, insert, delete, and a menu.

```
[1] ✓ 0s import re
```

```
[2] ✓ 0s texto = 'arara'
t = re.compile(r'a...a')
check = t.findall(texto)
print(check)
```

... ['arara']

<https://l1nq.com/sKMDL>

<https://regexr.com/>